

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Jasim Asmaan A	Reg. No.:	192421060
Date:	23-08-2026	Duration:	60 minutes

Code of Conduct

I Jasim Asmaan A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Jasim Asmaan A

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

The AI-Based Smart Disaster Alert and Emergency Response Coordination

1. Analyze the Project Requirements

1.1 Project Overview

The AI-Based Smart Disaster Alert and Emergency Response Coordination System is an intelligent emergency management platform designed to detect potential disasters, analyze disaster-related information, issue timely alerts, and coordinate emergency response activities.

The system uses Artificial Intelligence and real-time data to identify possible disaster situations and provide appropriate alerts to affected citizens and emergency response teams.

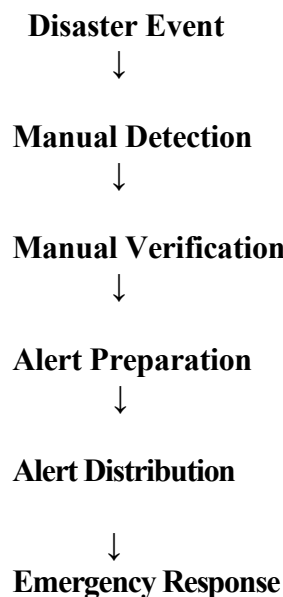
The system consists of:

- Citizen Application: For registration, location management, alerts and emergency assistance
- AI Disaster Detection Module: Analyzes incoming disaster-related data
- Alert Management Module: Generates and distributes disaster alerts
- Emergency Response Module: Coordinates rescue and emergency activities
- Location Management: Identifies affected geographical areas
- Emergency Dashboard: Provides real-time information to authorized responders
- Database: Stores user, disaster, location and response information

1.2 Why the System is Required

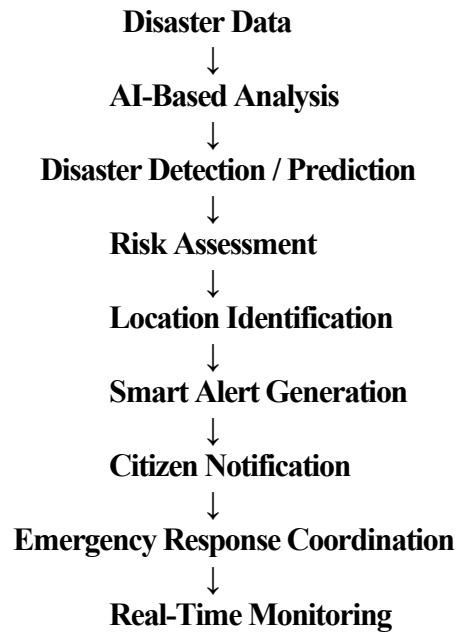
Traditional disaster response systems may depend on manual reporting and delayed communication.

Without an intelligent disaster management system:



This process can result in delays in identifying disasters and coordinating emergency services.

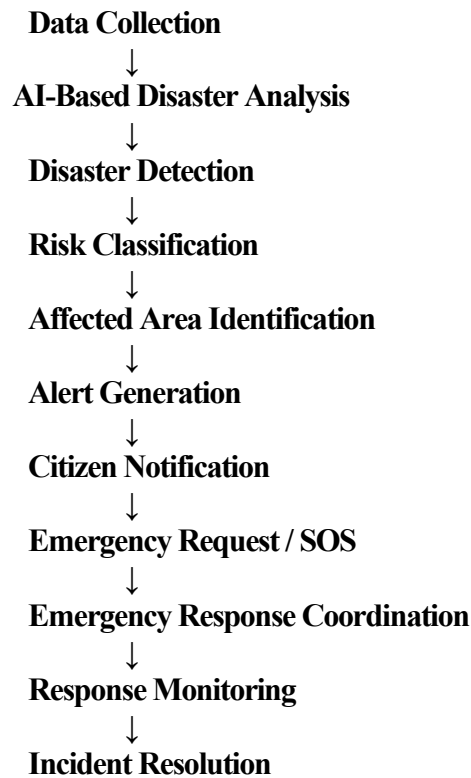
With the AI-Based Smart Disaster System:



The system enables faster communication between affected citizens and emergency response teams.

2. System Workflow

The proposed system contains the following stages:



System Stages

Data Collection Collect disaster-related information

Stage	Purpose
AI Analysis	Analyze incoming data using AI techniques
Disaster Detection	Identify possible disaster events
Risk Assessment	Determine severity and risk level
Location Analysis	Identify affected geographical areas
Alert Generation	Generate appropriate disaster warnings
Notification	Send alerts to affected citizens
Emergency Request	Allow citizens to request immediate assistance
Response Coordination	Coordinate emergency response teams
Monitoring	Track disaster and response status
Resolution	Record completion of emergency response

The system continuously processes disaster-related information and provides timely alerts and response coordination.

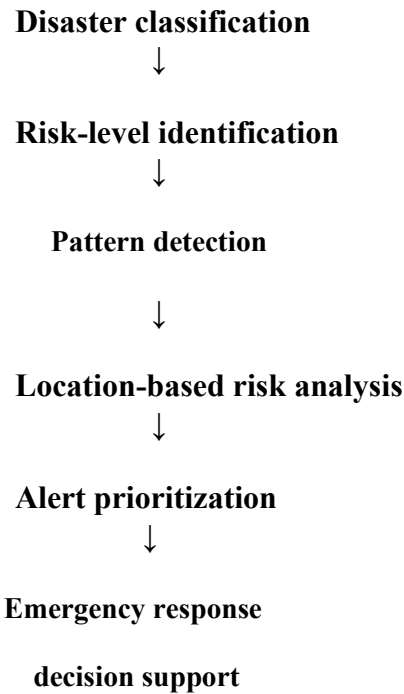
3.Tools and Technologies

The system can be developed using modern web, AI and database technologies.

Component	Technology
Frontend	React + Vite
Backend	Python + Flask
AI/ML	Python + Scikit-learn
Database	PostgreSQL
Backend	Python + Flask
Location Services	GPS / Maps API
Communication	REST API
Notifications	SMS / Push Notifications
Version Control	Git + GitHub
Containerization	Docker
Deployment	Docker Compose
Testing	Pytest / Vitest
Code Quality	ESLint / Flake8

Why AI is Used?

AI can help analyze disaster-related information and identify patterns that may indicate potential emergencies. The AI component can support:



4. Automated Testing Strategy

The system should automatically test critical functionalities before deployment.

```
(venv) PS E:\Projects\AI-Disaster-Alert-System\backend> python -m pytest ..\tests\test_alerts.py -v
===== test session starts =====
platform win32 -- Python 3.12.2, pytest-8.2.0, pluggy-1.5.0 -- E:\Projects\AI-Disaster-Alert-System\backend\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: E:\Projects\AI-Disaster-Alert-System\backend
collected 4 items

..\tests\test_alerts.py::test_disaster_detection PASSED [ 25%]
..\tests\test_alerts.py::test_alert_generation PASSED [ 50%]
..\tests\test_alerts.py::test_location_based_alert PASSED [ 75%]
..\tests\test_alerts.py::test_invalid_alert_request PASSED [100%]

===== 4 passed in 18.67s =====
(venv) PS E:\Projects\AI-Disaster-Alert-System\backend> █
```

Figure : Successful execution of automated backend tests using Pytest

4.1 Backend Testing

Use py test to test the backend APIs.

Example tests:

- Application starts successfully
- User registration works
- User login works
- Disaster alert API responds correctly
- Location update works
- Emergency/SOS request is processed
- Invalid requests are handled correctly
- Emergency response data is stored correctly

4.2 AI Model Testing

The AI component should be tested to verify:

- AI model loads successfully
- Disaster data is processed correctly
- Disaster category is predicted correctly
- Risk level is generated correctly
- Invalid input is rejected
- Prediction output is valid

- Model response time is acceptable

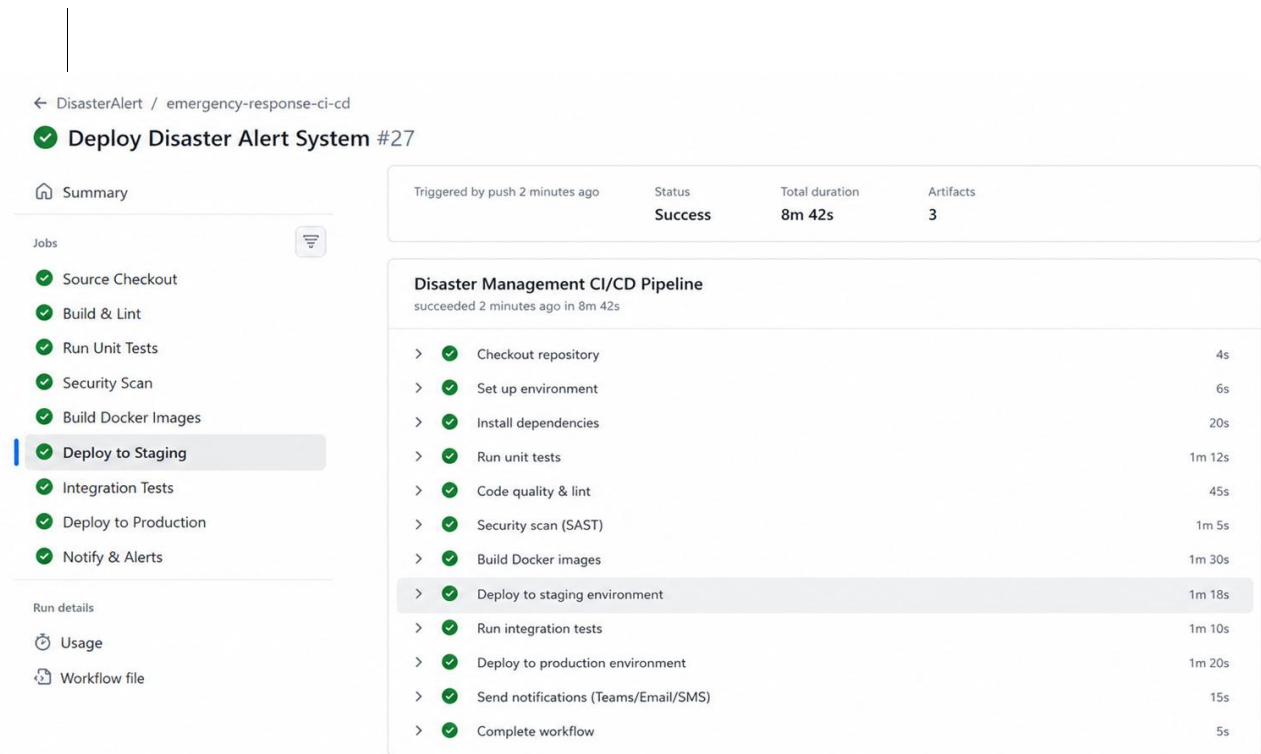


Figure: Automated disaster management pipeline execution in GitHub Actions

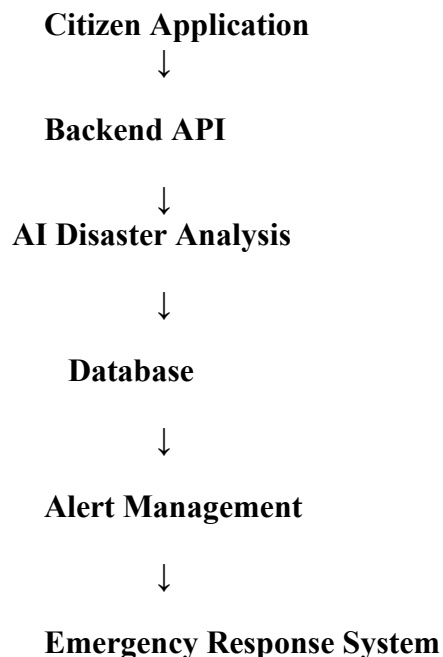
4.3 Frontend Testing

Automated frontend tests should verify:

- **Application renders correctly**
- **Registration and login work**
- **User location can be updated**
- **Disaster alerts are displayed**
- **Emergency/SOS button works**
- **Emergency information is displayed**
- **Map/location interface works**
- **Invalid input is handled correctly**
- **Emergency response status is displayed**

4.4 Integration Testing

Verify communication between:



The integration tests should ensure that all major modules communicate correctly.

4.5 Smoke Testing

After deployment:

Open Application



Homepage Loads?



Login Working?



Backend Responding?



AI Module Responding?



Alert API Responding?



Emergency Request Working?



PASS

If critical functionality fails, the deployment should be stopped or the previous stable version should be restored.

5. Deployment Strategy

Use three environments.

Development

Used by developers during implementation.

Developer



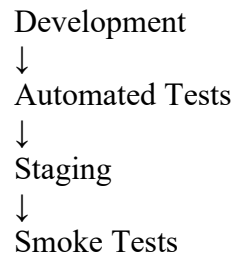
Feature Branch



Development Environment

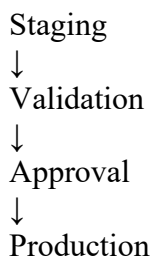
Staging

Used for final testing before production.

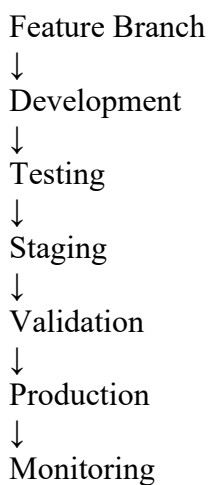


Production

The final environment used by citizens and authorized emergency response personnel.



Environment Flow



The production system should be continuously monitored because disaster alerts and emergency coordination are critical services.

6. Security, Quality and Recovery

6.1 Security

The system should include:

- Secure user authentication
- Role-based access control
- Password protection
- HTTPS communication
- Encryption of sensitive information
- Secure API access
- Secure database access
- Protection of user location information
- Protection against unauthorized access
- Secure management of API keys
- No passwords or secrets stored in source code
- Regular security testing

6.2 Quality Checks

Before deployment:

Code Review

↓

Linting

↓

Unit Tests

↓

Integration Tests

↓

AI Model Validation

↓

Security Testing

↓

System Build

If any critical check fails:

System Validation → FAILED



Deployment STOPPED

7. Recovery Mechanism

If the new production version fails, the system should restore the previous stable version.

Example:

System Version 1.0



Production



Version 1.1 Deployed



System Failure



Recovery / Rollback



Version 1.0 Restored

The recovery mechanism reduces system downtime and helps maintain the availability of critical disaster alert and emergency response services.

A backup mechanism should also be maintained for important disaster, user and emergency response data.

System Diagram

Figure: AI-Based Smart Disaster Alert and Emergency Response Coordination System

Disaster Data Sources



AI-Based Analysis



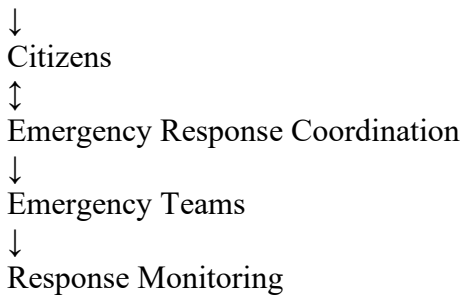
Disaster Detection & Risk Assessment



Location Identification



Alert Management

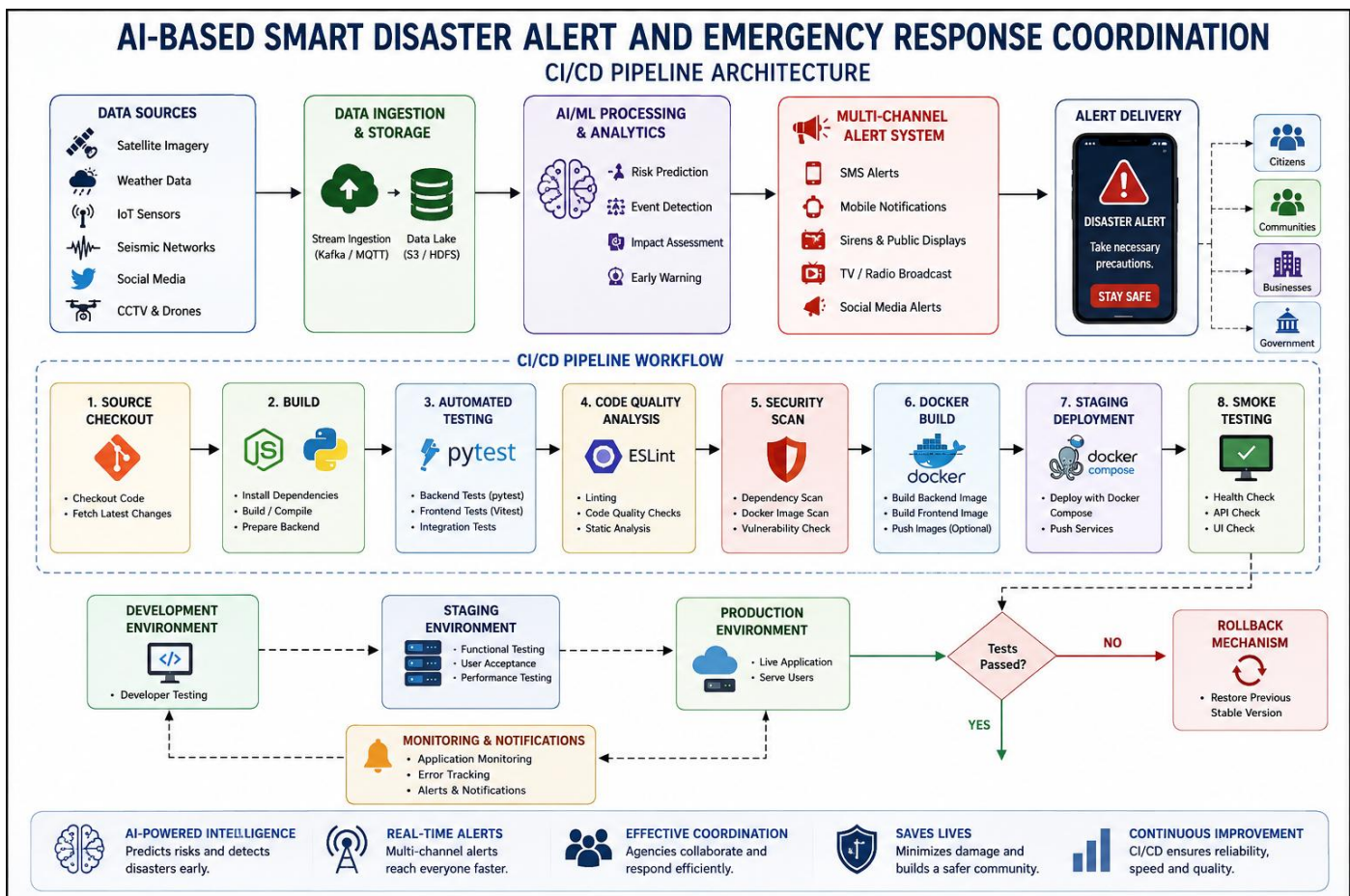


The proposed system integrates AI-based disaster analysis, location identification, alert generation and emergency response coordination.

The AI module analyzes disaster-related information and identifies potential risks. Based on the identified risk and affected location, the system generates appropriate alerts for citizens.

Citizens can receive warnings and submit emergency requests when assistance is required. Emergency response teams can use the coordination module to monitor incidents and manage response activities.

Figure: Overall Workflow of the AI-Based Smart Disaster Alert and Emergency Response Coordination System



Data Collection → AI Analysis → Risk Assessment → Alert → Citizen → Emergency Request → Response Team → Resolution

AI-Based Disaster Analysis

The AI module analyzes the available disaster information and identifies possible disaster conditions.

Risk Classification

The system classifies the identified event according to its severity and risk level.

Affected Area Identification

The system identifies the geographical area that may be affected.

Smart Alert Generation

The system generates a suitable warning based on disaster type, severity and affected location.

Citizen Notification

The alert is delivered to citizens in the affected area.

Emergency Request

Citizens can send an emergency request or SOS when assistance is required.

10. Workflow Explanation

Step 1 — User Registration

The citizen creates an account and securely logs into the system.

Step 2 — Location Update

The citizen's current location is captured or updated for location-based disaster alerts.

Step 3 — Data Collection

The system collects relevant disaster-related information.

Step 4 — AI Analysis

The AI module analyzes the incoming information and identifies potential disaster events.

Step 5 — Risk Assessment

The system determines the risk level and severity of the identified event.

Step 6 — Alert Generation

An appropriate disaster alert is generated for the affected location.

Step 7 — Notification

Citizens in the affected area receive the disaster warning.

Step 8 — Emergency Request

Citizens can submit an emergency request or SOS when immediate assistance is required.

Step 9 — Response Coordination

Emergency response teams receive emergency requests and coordinate appropriate response activities.

Step 10 — Response Assignment

Emergency requests can be assigned to appropriate response personnel or teams.

Step 11 — Monitoring

The system monitors the status of disaster incidents and emergency responses.

Step 12 — Resolution

Once the emergency response is completed, the incident is marked as resolved and the information is recorded.

11. Justification

The proposed AI-Based Smart Disaster Alert and Emergency Response Coordination System is appropriate because it combines Artificial Intelligence, real-time information, location-based services and emergency coordination to improve disaster management.

The AI component supports early identification and classification of potential disaster situations. Location-based alerts help ensure that warnings reach citizens in affected areas.

The emergency coordination module enables response teams to receive and manage emergency requests efficiently.

The proposed system improves:

- Early disaster detection through AI-based analysis
- Alert delivery through location-based notifications
- Public safety through timely disaster warnings
- Emergency response through coordinated rescue activities
- Decision support through AI-based risk assessment
- Response efficiency through emergency request management

- Real-time monitoring through centralized dashboards
- Reliability through systematic testing
- Security through authentication and access control
- Data protection through secure communication and storage
- Scalability through modular system architecture
- Maintainability through structured development
- Availability through monitoring and recovery mechanisms

Conclusion

The AI-Based Smart Disaster Alert and Emergency Response Coordination System provides an intelligent approach to disaster management by combining AI-based analysis, location-aware alerts, emergency requests and coordinated response activities.

The system can help improve the speed, reliability and effectiveness of communication between disaster information sources, citizens and emergency response teams.